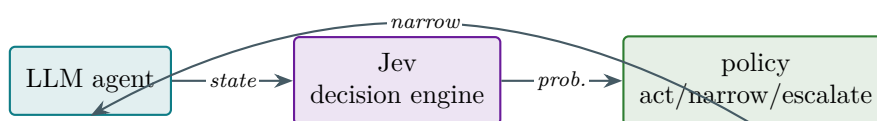


The ARBITER Handbook

A build-along guide to calibrated decisions for LLM agents



A living companion to `docs/PLAN.md` and the four research docs.
Every figure states where its numbers come from. No illustrative value is a result.

Last built: September 27, 2026

Contents

How to read this book (by goal)	vii
I. The big picture	1
1. What we are building	3
1.1. The problem in one paragraph	3
1.2. The idea	3
1.3. Why this and not something else	3
1.4. What it is not	4
2. The roadmap	5
2.1. Milestones and gates	5
2.2. Estimated timeline	5
2.3. How the pieces depend on each other	5
II. Foundations	7
3. LLM agents and tool calling	9
3.1. The agent loop	9
3.2. Tool calling guarantees shape, not confidence	9
3.3. Where decisions live at scale	9
4. Jev and “System One” models	11
4.1. What Jev does, in one line	11
4.2. The three primitives	11
4.3. Anatomy of a request and response	11
4.4. The nine documented failure modes	12
5. Probability, confidence, and calibration	13
5.1. Calibration, defined	13
5.2. Measuring miscalibration: ECE and the noise floor	13
5.3. Risk–coverage: the operating decision	13
5.4. Fixing calibration after the fact	14
5.5. Conformal prediction (a brief note)	15
6. Routers, cascades, and verifiers	17
6.1. Routers	17
6.2. Cascades	17
6.3. Verifiers and the “information the proposer lacks” rule	18
7. Evaluating agents honestly	19
7.1. Reliability: $\text{pass}@k$ vs pass^k	19
7.2. Cost per successful task and the Pareto frontier	19
7.3. The controls that stop us fooling ourselves	19

III. The system	21
8. Decision points DP1–DP7	23
8.1. The design rule	23
8.2. The two MVP decisions	23
9. SDK architecture	25
9.1. Components	25
9.2. One loop iteration (MVP)	25
9.3. A single decision, in sequence	25
9.4. Confidence bands	25
9.5. Security boundary	25
9.6. Secrets: allow-list by reference	25
10. Engineering practices	29
10.1. Record / replay cache	29
10.2. Version pinning	29
10.3. The test pyramid	29
IV. Evidence so far	31
11. Evidence so far	33
11.1. Accuracy: competitive on short tasks, weak on long ones	33
11.2. Latency: fast, but with tails under load	33
11.3. Calibration	33
11.4. Where the hypotheses stand	34
V. Build log	35
12. Build log	37
VI. Appendices	39
A. Glossary	41
B. Formula sheet	43
C. Answers to “Check yourself”	45
D. Source map	47

List of Figures

1.1.	The arbiter value loop: state becomes a small projection, a cheap engine returns probabilities, and a deterministic policy acts, narrows, or escalates. [Data: Illustrative]	4
2.1.	Milestone flow. Gates G0/G1/G2 are go/modify/stop checkpoints. Product surface (M3) and evidence (M5/M6) both start after G0. [Data: Illustrative]	5
2.2.	Estimated schedule to the M7 evidence release (about 20–30 weeks). [Data: Illustrative (estimate, docs/PLAN.md §6.2)]	6
2.3.	Dependency graph: arrows point from a component to what it enables. Offline components (left) come before networked engines. [Data: Illustrative (docs/IMPLEMENTATION_PLAN.md §2)]	6
3.1.	The agent loop. The decision layer intercepts the “LLM decides” step for bounded, closed-set decisions. [Data: Illustrative]	9
4.1.	The three Jev primitives and what each returns. [Data: Reported by TypeSafe docs] [18]	11
4.2.	The nine documented failure modes and what each forces in the architecture. [Data: Reported by TypeSafe docs] [16]	12
5.1.	A reliability diagram. Points on the dashed line are calibrated; below it is overconfident, above it is underconfident. [Data: Illustrative (synthetic data)]	13
5.2.	Jev’s ECE is near the noise floor in distribution but about $4.4\times$ the floor out of distribution. [Data: Reported by jev-ood-calibration] [15]	14
5.3.	A risk–coverage curve: acting only on the most confident decisions raises accuracy on the automated fraction. [Data: Illustrative (synthetic data)]	14
5.4.	Post-hoc recalibration pulls an overconfident model toward the diagonal. [Data: Illustrative (synthetic data), effect direction reported by Jev-Calibration] [2]	15
6.1.	Router vs cascade. A router decides <i>before</i> paying (top); a cascade pays the cheap model first, then defers on low confidence (bottom). [Data: Illustrative]	17
7.1.	For a per-attempt success of 0.7, $\text{pass}@k$ rushes to 1 while pass^k collapses. Reliability is the honest metric. [Data: Exact (computed from $p = 0.7$)]	19
7.2.	Illustrative accuracy–cost plot. The question is whether the hybrid (A1) sits up-and-left of the baseline (A0) and beats the random control (A5). [Data: Illustrative (synthetic data); arm definitions from docs/PLAN.md §9]	20
8.1.	The decision-point taxonomy. The MVP uses DP2 and DP4 (bold); DP3 is shadow-only, the rest are deferred. [Data: Reported by docs/ARCHITECTURE.md §3]	23
8.2.	Authority classes: the riskier the consequence, the less the engine may decide on its own. [Data: Illustrative (docs/ARCHITECTURE.md §3, §10)]	23
9.1.	SDK components. The engine is advisory; the policy narrows or escalates; code owns execution and safety. [Data: Illustrative (docs/ARCHITECTURE.md §4, docs/PLAN.md §4)]	26
9.2.	The MVP loop: DP4 monitors progress, DP2 narrows the tool, the LLM writes arguments, and code validates and executes. [Data: Illustrative (docs/PLAN.md §5)]	26

List of Figures

9.3.	Sequence for one DP2 decision: projection out, probabilities back, policy picks a band, the LLM fills arguments. [Data: Illustrative (docs/ARCHITECTURE.md §5.3)] . .	26
9.4.	The policy maps the gating statistic to three bands. Thresholds t_{esc}, t_{act} are fitted per DP, primitive, and model version. [Data: Illustrative (docs/ARCHITECTURE.md §6)]	26
9.5.	Advisory vs authoritative layers. Injection is assumed; the engine is never the boundary. [Data: Illustrative (docs/ARCHITECTURE.md §10)]	27
10.1.	Record/replay: on a miss, call the API and store; on a hit, replay. The hash excludes wall-clock and counters so it is stable across processes. [Data: Illustrative (docs/PLAN.md EF-11)]	29
10.2.	The test pyramid for the harness. Contract tests keep engines interchangeable; fault injection proves the fallback path. [Data: Illustrative (docs/PLAN.md §7.3)] . . .	30
11.1.	Jev is within a few points of the comparator LLM on short-input intent classification, clearly below on whole-document phishing, and recovers most of the gap when the judgment is decomposed. The comparator is the frontier model GPT-5.6 Terra on Banking77/CLINC150 and Claude Haiku 4.5 on phishing; the encoder is bge-small+logistic regression on Banking77 and a 4B LoRA model on phishing (no published CLINC150 encoder number, so that bar is omitted). A trained in-domain encoder tends to win when labels exist. [Data: Reported by jev-baselines-eval and jev-phishing-bench] [1, 7]	33
11.2.	Illustrative latency profile: sub-second typical, multi-second tails under heavy concurrency. [Data: Illustrative; ranges reported by Aman Kumar] [10]	34
11.3.	Hypothesis status at the start of the build. The gates G0/G1/G2 are designed to move these. [Data: Reported by docs/DECISION_LOG.md §1]	34

How to read this book (by goal)

This book teaches everything behind the **arbiter** project: what Jev is, how calibration works, how the SDK is designed, and what the evidence says. It is a living document, rebuilt as the project progresses (see the Build log in Part V).

Reading paths by goal

10-minute tour

Chapter 1 (what we build) then Chapter 4 §4.1 (what Jev does in one line) then Figure 9.1 (the architecture).

SDK user

Part II foundations, then Chapter 9 (architecture) and Chapter 8 (decision points).

Researcher

Chapter 5 (calibration), Chapter 7 (evaluating agents), and Part IV (evidence).

Contributor

Chapter 10 (engineering practices) and the Build log (Part V).

Conventions

Every factual claim carries an evidence label so you always know how much to trust it: [DOC] verified in TypeSafe’s documentation; [IND] an independent measurement; [VENDOR] a vendor claim; [HYP] our own hypothesis; [MEASURED] measured by us (with a run ID). No [MEASURED] labels exist yet, because we have not run experiments.

Every figure and table caption ends with a **data label** in brackets stating where its numbers come from: *Illustrative* (a drawing, not data), *Exact* (computed from a formula), *Reported by ...* (someone else’s measurement, cited), or *Measured by us*. This rule exists so that a teaching diagram is never mistaken for a result.

Key idea

The whole project rests on one honest distinction: a model’s *answer* and its *calibrated confidence in that answer* are two different things. Most of this book is about the second one.

Part I.

The big picture

1. What we are building

We are building **arbiter** (ARBITER: Agent Runtime for Bounded Inference, Triage, Evaluation & Routing): an open-source Python SDK that adds *typed, calibrated decision points* to an LLM agent a developer already runs (LangChain, Pydantic AI, or the OpenAI Agents SDK).

1.1. The problem in one paragraph

An LLM agent makes many small, bounded decisions on every loop: which tool to call next, whether the task is finished, whether it is stuck. Today the same expensive model that plans and writes text also makes these decisions. That is slow, costly, and, worst of all, the decision comes back as plain text with *no usable number* telling you how sure the model is. You cannot tell a confident choice from a coin flip.

1.2. The idea

At each decision point the SDK does four things (Figure 1.1):

1. Build a small, token-budgeted **projection** of the agent's state.
2. Ask a cheap, swappable **engine** (Jev by default) a bounded typed question and get back *probabilities*.
3. A deterministic **policy** reads the (recalibrated) probability and either **acts**, **narrows** the LLM's options, or **escalates** to the LLM or a human.
4. Log the decision and its outcome so thresholds can be fitted and results reproduced.

1.3. Why this and not something else

The research phase (Part IV) reached a clear verdict: integrations already exist (LangChain, Pydantic AI, and Vercel all ship Jev support, and about 200 community projects exist [11, 13]). What nobody has published is *controlled evidence*, with proper baselines and trajectory-level calibration, that inserting a calibrated decision component actually lowers cost per successful task without hurting reliability. So the project has two co-equal deliverables:

Deliverable A — the SDK

the instrument developers install.

Deliverable B — the evidence

a pre-registered, reproducible study (with a released decision dataset and trajectory-checkpoint calibration) that no one else has.

1. What we are building

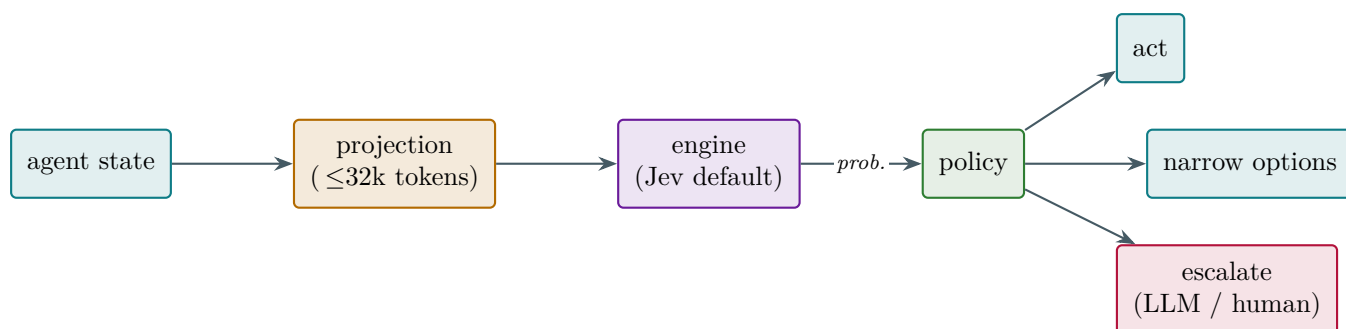


Figure 1.1.: The arbiter value loop: state becomes a small projection, a cheap engine returns probabilities, and a deterministic policy acts, narrows, or escalates. [Data: Illustrative]

Why it matters here

The engine is *swappable* on purpose. If Jev turns out not to help on your workload, you change one line to an LLM-backed engine, a small trained classifier, or the open-weights Laya model [4], and the SDK still works. The product survives a negative result about any single engine.

1.4. What it is not

It is not a new agent framework (that is an explicit non-goal). It does not plan, write tool arguments, or generate text; the LLM keeps those jobs. It is never the security or authorization boundary; deterministic code owns that. The MVP is deliberately small: two decision points (which tool, and done/stuck) on one host framework.

Check yourself

Why can a model's answer be right while its confidence is untrustworthy? What would you measure to catch that? (Answer in Appendix C.)

2. The roadmap

The project is a sequence of milestones M0–M8, each mapped to a phase of the research roadmap and separated by **gates** (go/modify/stop checkpoints). The rule is *evidence before claims*: no milestone publishes a cost or reliability number before the gate that produced it.

2.1. Milestones and gates

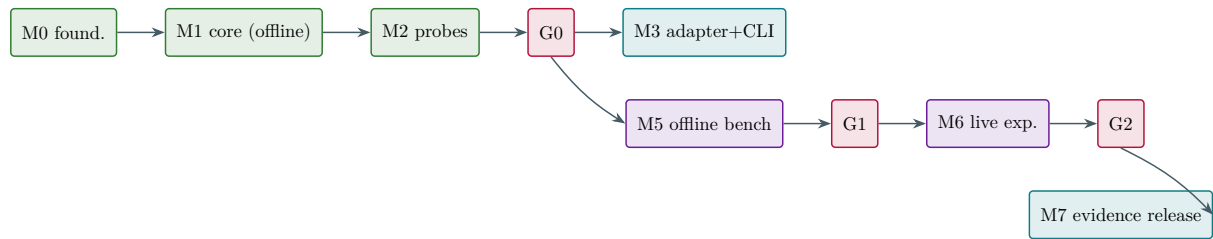


Figure 2.1.: Milestone flow. Gates G0/G1/G2 are go/modify/stop checkpoints. Product surface (M3) and evidence (M5/M6) both start after G0. [Data: Illustrative]

Milestone	Maps to	Ends with
M0 Foundations	(setup)	this session’s docs
M1 Core SDK (offline)	research P0/P1 eng.	green offline tests, no keys
M2 Engines + probes	research P0	Gate G0
M3 Adapter + CLI	(product)	quickstart under 5 min from cache
M4 Public alpha	(product, optional)	v0.1, no claims
M5 Offline benchmark	research P1	Gate G1
M6 Live experiment	research P2	Gate G2
M7 Evidence release	write-up	reproducible results page
M8 Extensions	research P3	(optional)

Table 2.1.: Milestones mapped to the frozen research phases. [Data: Reported by docs/PLAN.md §6]

2.2. Estimated timeline

Dates are estimates, re-planned at each gate. Most of the calendar is API runs, human labeling, and gate reviews, not coding.

2.3. How the pieces depend on each other

Build the thing with the fewest dependencies first. The offline core (log, cache, projections, policy, calibrators) needs no API keys, so it is built and tested before any engine that calls a network service.

2. The roadmap

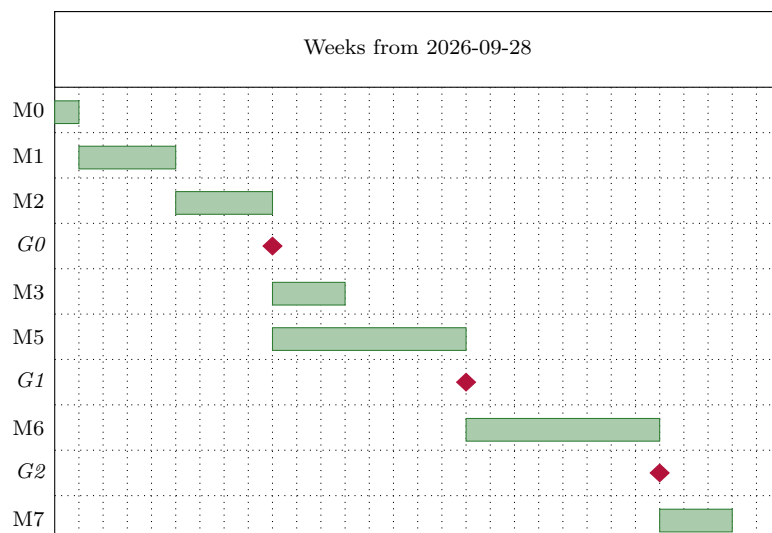


Figure 2.2.: Estimated schedule to the M7 evidence release (about 20–30 weeks). [Data: *Illustrative (estimate, docs/PLAN.md §6.2)*]

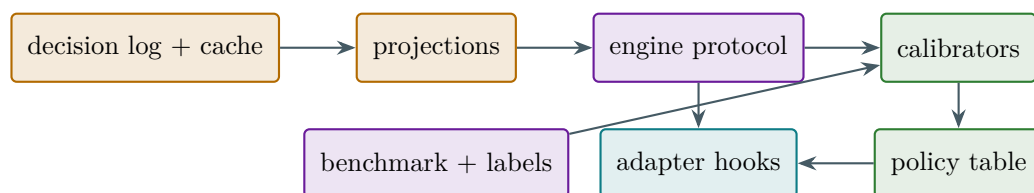


Figure 2.3.: Dependency graph: arrows point from a component to what it enables. Offline components (left) come before networked engines. [Data: *Illustrative (docs/IMPLEMENTATION_PLAN.md §2)*]

Check yourself

Why build the record/replay cache before any real engine? What does it buy you at gate review time? (Appendix C.)

Part II.

Foundations

3. LLM agents and tool calling

An **LLM agent** is a loop: the model observes state, decides on an action (usually a tool call), the runtime executes it, and the observation feeds back in. Everything the decision layer does hangs off this loop, so it is worth being precise about it.

3.1. The agent loop

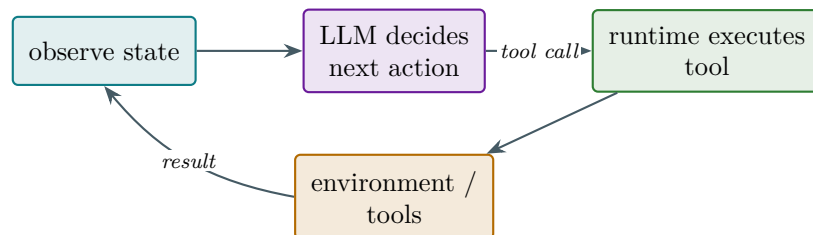


Figure 3.1.: The agent loop. The decision layer intercepts the “LLM decides” step for bounded, closed-set decisions. [Data: Illustrative]

3.2. Tool calling guarantees shape, not confidence

Modern APIs let the model emit a structured **tool call** (a function name plus JSON arguments). Structured outputs and strict tool use guarantee the *shape* is valid, but hosted APIs generally do not return a probability distribution over *which* tool was chosen. That missing distribution is exactly the primitive the decision layer supplies.

Key idea

Frameworks already expose a “pre-action” seam: a callable that returns an enum (allow/deny/ask) or a node name. Escalation there is a boolean, never a probability. The decision layer plugs into that seam and makes the signal a calibrated probability instead.

3.3. Where decisions live at scale

When a catalog has dozens or hundreds of tools, accuracy degrades: vendors themselves note tool selection gets worse past roughly 30–50 tools, which is why they ship tool-search and allowed-tools features. Narrowing the catalog *before* the LLM chooses is a real job, and a cheap classifier is a natural fit for it. That is decision point DP2, the MVP’s first target (Chapter 8).

Pitfall

A tempting shortcut is to let the model both narrow and choose in one call. That reintroduces the cost you were trying to avoid and hides the decision inside free text, where you cannot attach a confidence or test it. Keep the bounded decision separate and typed.

3. *LLM agents and tool calling*

Check yourself

Name two things a hosted tool-calling API guarantees and one thing it does not. Which of the three does the decision layer add? (Appendix [C](#).)

4. Jev and “System One” models

4.1. What Jev does, in one line

Jev reads a text or JSON **state** and answers typed, closed questions about it, returning probabilities and (for two of the three question types) a confidence statistic. It does not generate text.^[DOC] [19]

TypeSafe calls it a **System One** model, after Kahneman’s fast, intuitive “System 1”: it makes quick bounded judgments, not deliberate multi-step reasoning.

4.2. The three primitives

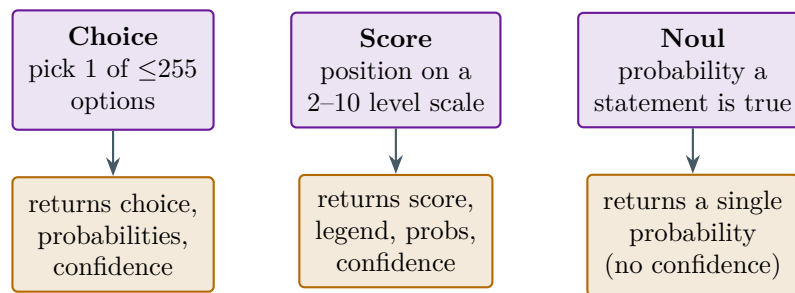


Figure 4.1.: The three Jev primitives and what each returns. [Data: Reported by TypeSafe docs] [18]

Choice one of up to 255 unordered options; returns the pick, a probability per option, and a confidence.^[DOC]

Score a position on an ordered 2–10 level scale; the score is a probability-weighted mean of level numbers. Numerical interpolation is weakly calibrated, so use scores for thresholds, not arithmetic.^[DOC]

Noul the probability that a statement is true; no separate confidence field.^[DOC]

4.3. Anatomy of a request and response

You send one *state* and a set of independent *questions*; they are evaluated in parallel and cannot see each other’s answers.^[DOC] Dependent questions need a second request.

Listing 4.1: A Choice request and its response (schematic).

```
1 // request
2 { "state": { "task": "book a flight", "last_actions": ["search_flights"] },
3   "model": "jev-1.13.0",
4   "questions": [ { "id": "next_tool", "type": "choice",
5                   "options":
6                     ["select_seat", "pay", "search_flights", "none_of_the_above"] } ] }
6 // response
```

4. Jev and “System One” models

```
7 { "answers": { "next_tool": { "choice": "select_seat",  
8   "probabilities": {"select_seat":0.71,"pay":0.20,"search_flights":0.08,  
9     "none_of_the_above":0.01},  
10   "confidence": 0.68 } },  
11   "model": "jev-1.13.0", "usage": {"input_tokens": 41} }
```

Worked example

Cost is dominated by input tokens: at the list price of \$0.042 per million input tokens with free output, 20,000 decisions of about 1,500 input tokens each cost roughly \$1.26.^[DOC] [17] The limits that shape the SDK: at most 255 Choice options, 2–10 Score levels, and 64k tokens per request (32k for state plus the longest question).^[DOC]

4.4. The nine documented failure modes

TypeSafe publishes a “jaggedness” page listing where Jev fails and how to work around it.^[DOC] [16] These are load-bearing for the design.

Failure mode	Consequence for the design
1. Literal reading	Question wording becomes a versioned artifact.
2. Math / counting	Keep arithmetic and counts in code.
3. Date/time comparison	Extract components; compute dates in code.
4. Indirection / multi-hop	Excludes planning-level decisions.
5. Large irrelevant state	Send projections, never full history.
6. Adversarial content	Cannot be the security boundary.
7. Contradictory criteria	Criteria design discipline.
8. No cross-primitive invariants	Do not transfer thresholds between primitives.
9. No generation	The LLM writes all text and arguments.

Figure 4.2.: The nine documented failure modes and what each forces in the architecture. [Data: Reported by TypeSafe docs] [16]

Why it matters here

Every one of these nine drives a concrete rule later: projections (mode 5), deterministic verifiers (modes 2–3), advisory-only authority (mode 6), frozen question wording (mode 1), and per-primitive thresholds (mode 8). The design is essentially a disciplined response to this table.

Check yourself

Jev returns a probability of 0.99 that a shell command is safe. Give two reasons from this chapter why code, not Jev, must still decide whether to run it. (Appendix C.)

5. Probability, confidence, and calibration

This is the conceptual center of the project. A model can be accurate yet *miscalibrated*: its stated probabilities do not match how often it is right. Gating actions on an untrustworthy probability is worse than not gating at all.

5.1. Calibration, defined

A model is **calibrated** if, among all the times it says “0.8”, it is right about 80% of the time. This is a property of *groups* of predictions, not any single one. TypeSafe states calibration exactly this way and warns it “is not a guarantee about any single answer.”^[DOC]

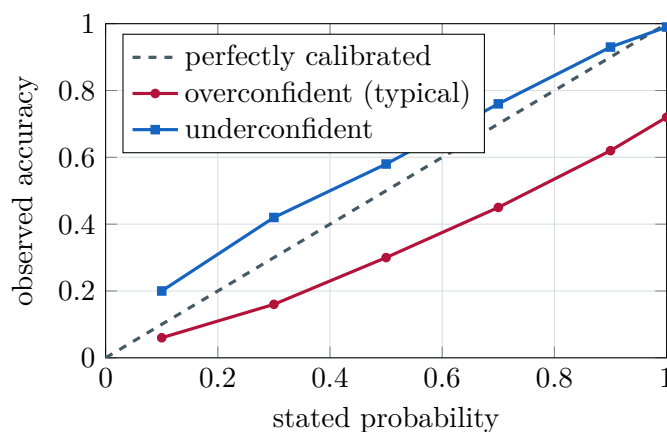


Figure 5.1.: A reliability diagram. Points on the dashed line are calibrated; below it is overconfident, above it is underconfident. [Data: Illustrative (synthetic data)]

5.2. Measuring miscalibration: ECE and the noise floor

The **Expected Calibration Error (ECE)** bins predictions by stated probability and averages the gap between confidence and accuracy in each bin, weighted by bin size:

$$\text{ECE} = \sum_{b=1}^B \frac{n_b}{N} |\text{acc}(b) - \text{conf}(b)|.$$

A subtlety that honest evaluations respect: with finite samples, even a perfectly calibrated model shows a small nonzero ECE. That is the **noise floor**. Reporting ECE without its floor overstates miscalibration.

5.3. Risk–coverage: the operating decision

Gating is a trade: act only when confident (high **coverage** costs accuracy; low coverage buys it). A **risk–coverage curve** shows accuracy on the automated fraction as you sweep the threshold. The product’s G0 gate asks for coverage $\geq 50\%$ at a chosen precision target.

5. Probability, confidence, and calibration

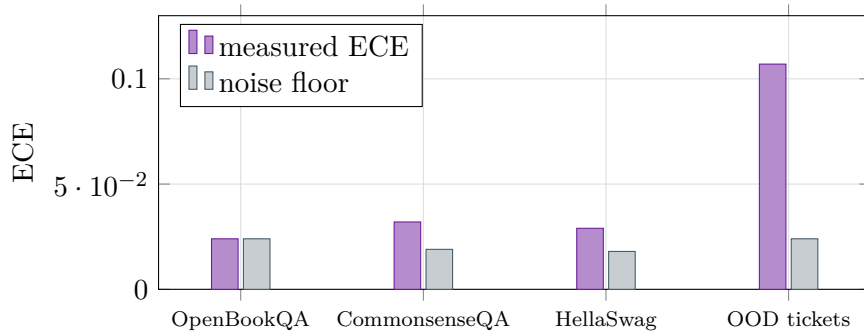


Figure 5.2.: Jev’s ECE is near the noise floor in distribution but about $4.4\times$ the floor out of distribution. [Data: Reported by *jev-ood-calibration*] [15]

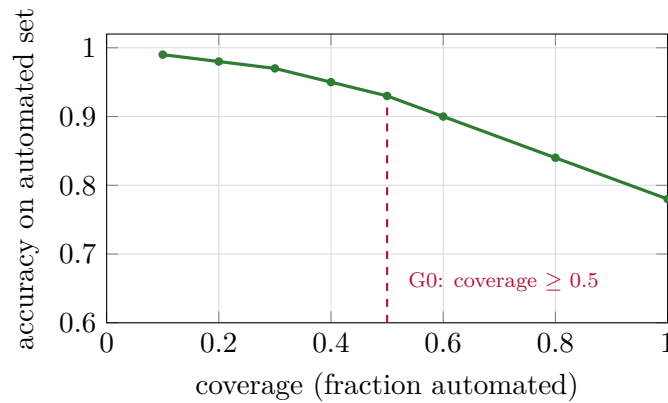


Figure 5.3.: A risk–coverage curve: acting only on the most confident decisions raises accuracy on the automated fraction. [Data: Illustrative (synthetic data)]

5.4. Fixing calibration after the fact

Two standard post-hoc recalibrators, fitted on a held-out labeled split:

Temperature scaling

divide the logits by a single learned scalar T before the softmax [6]. Simple, but it needs *logits*. If the API returns only probabilities, or a probability is exactly 1.0, temperature scaling is undefined ($\log 1 = 0$ after the softmax, log of a saturated prob is problematic). This is a real constraint for Jev (see the pitfall).

Isotonic regression

fit a monotonic step function from stated probability to observed frequency. Works on probabilities directly and handled Jev well in an independent study, reaching ECE ≈ 0.01 with a few hundred labels [2].

Pitfall

Independent testers found Jev returns probability *exactly* 1.0 on many items, some of them wrong [7]. A saturated 1.0 breaks temperature scaling and collapses isotonic bins. The SDK’s rule: verify whether logits are available; if not, use isotonic or histogram binning with epsilon-clipping, and add a “saturated” band that *always escalates* for any authority above a plain filter. “1.0 is not proof.”

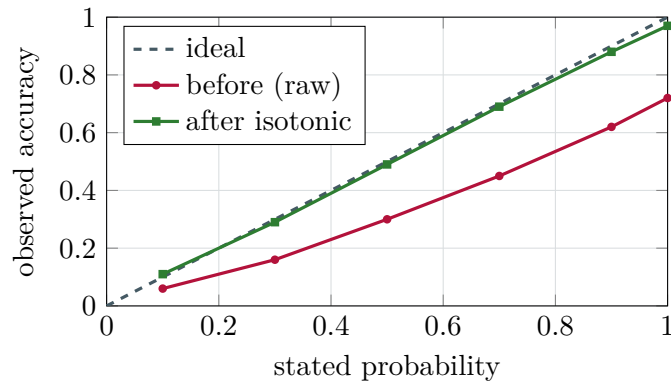


Figure 5.4.: Post-hoc recalibration pulls an overconfident model toward the diagonal. [Data: *Illustrative (synthetic data)*, effect direction reported by *Jev-Calibration*] [2]

Why it matters here

Independent studies also found the vendor **confidence** field was *less* calibrated than the maximum option probability [15]. So the SDK defaults to gating on max probability, logs both, and lets the G0 study pick the winner. Thresholds are fitted per decision point, per primitive, per model version, and refit when wording, catalog, or version changes.

5.5. Conformal prediction (a brief note)

Conformal prediction builds a set of options guaranteed (under exchangeability) to contain the truth with a chosen probability, using a calibration set. Robotics work (KnowNo) used it to decide when an agent should ask for help [14]. It is a natural future tool for the escalation policy; the MVP starts with simpler threshold gating.

Check yourself

A model scores 92% accuracy and ECE 0.15. Your policy acts whenever confidence ≥ 0.9 . Why might this be dangerous, and which figure in this chapter would you inspect first? (Appendix C.)

6. Routers, cascades, and verifiers

The decision layer sits in a long line of ideas for spending less compute without losing quality. Knowing where they succeed, and where they usually fail, keeps our claims honest.

6.1. Routers

A **router** sends each query to the cheapest model likely to handle it. RouteLLM trains strong-vs-weak routers from preference data and reports large savings at matched quality [12]. The cautionary result: benchmarks found many routers barely beat a trivial “mix models at random in the right proportion” baseline, and routers trained on one distribution were near-random on another. Baselines matter.

6.2. Cascades

A **cascade** tries a cheap model first and escalates to an expensive one when a reliability score is low. FrugalGPT is the canonical example [3]. But theory shows confidence-based deferral *provably fails* when the downstream model is a specialist, under label noise, and under distribution shift [8].

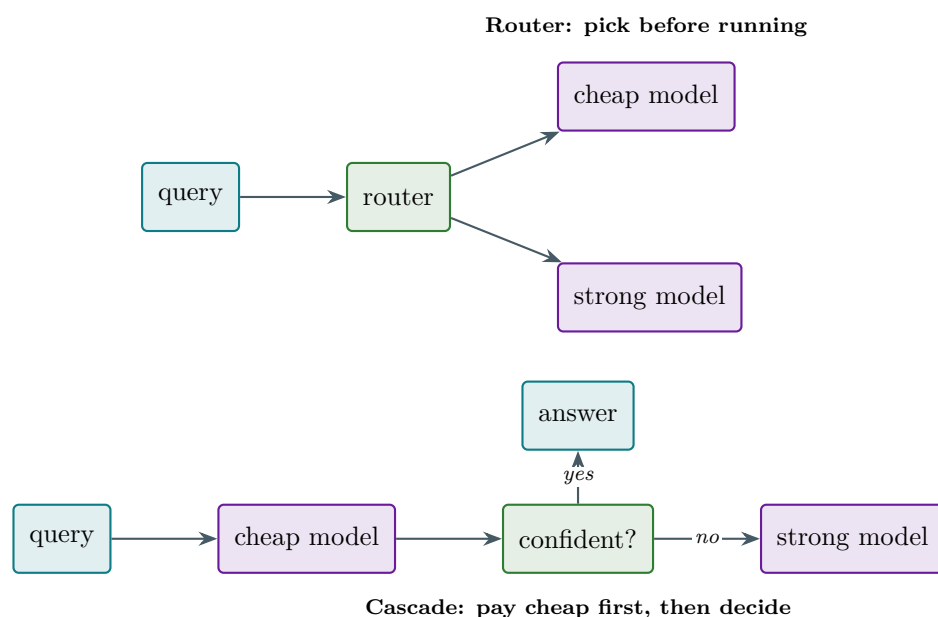


Figure 6.1.: Router vs cascade. A router decides *before* paying (top); a cascade pays the cheap model first, then defers on low confidence (bottom). [Data: Illustrative]

Key idea

Because Jev is so cheap, it acts more like a *pre-generation router* (decide before paying for the LLM) than a cascade rung. What decides whether it helps is not its latency but its *error rate on the specific decision*, and whether its confidence is calibrated enough to

gate on.

6.3. Verifiers and the “information the proposer lacks” rule

Separating a **proposer** (the LLM) from a **scorer/verifier** helps in robotics and web agents, but the literature is consistent about *why*: it helps when the scorer has information the proposer does not, such as execution results or environment state, not merely because it is a second model. Imperfect verifiers also cap the gains and can be gamed at large sample counts.

Why it matters here

This sets our expectation for the honest outcome: “cheaper decisions at equal or slightly lower accuracy,” not “a strictly better agent.” The evaluation is designed to detect exactly that, and the arms (Chapter 7) separate “typed decision helped” from “Jev specifically helped” from “more compute helped.”

Check yourself

Why is a cheap, calibrated classifier better modeled as a pre-generation router than as a cascade rung? (Appendix C.)

7. Evaluating agents honestly

An impressive demo proves nothing. This chapter is the measurement discipline that makes the project’s claims trustworthy, drawn from the agent-evaluation literature.

7.1. Reliability: $\text{pass}@k$ vs pass^k

$\text{pass}@k$ asks whether the agent succeeds in *at least one* of k tries; it flatters flaky agents. pass^k asks whether it succeeds in *all* k tries; it measures reliability [20]. As k grows the two diverge sharply for an unreliable agent.

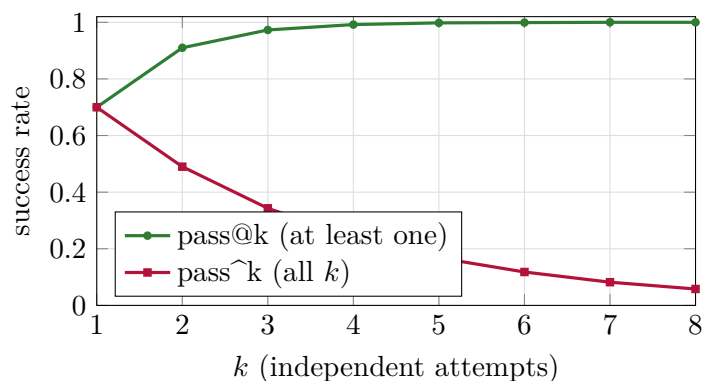


Figure 7.1.: For a per-attempt success of 0.7, $\text{pass}@k$ rushes to 1 while pass^k collapses. Reliability is the honest metric. [Data: Exact (computed from $p = 0.7$)]

7.2. Cost per successful task and the Pareto frontier

Accuracy alone hides the bill. The right object is the **accuracy–cost Pareto frontier**: plot each system by cost and success, and compare frontiers, not single points [9]. A famous result: a plain “retry” baseline beat an elaborate agent at a fraction of the cost, which is why baselines like ARM-A5 (random) and ARM-A6 (matched compute) exist in our design.

7.3. The controls that stop us fooling ourselves

Same-interface baseline (ARM-A2)

an LLM answering the *same* typed questions, to separate “typed decision points help” from “Jev specifically helps.”

Matched compute (ARM-A6)

give the baseline the same extra budget the hybrid spends, so a win is not just “more compute.”

Random policy (ARM-A5)

the same machinery making random decisions, to control for the structure of the intervention.

7. Evaluating agents honestly

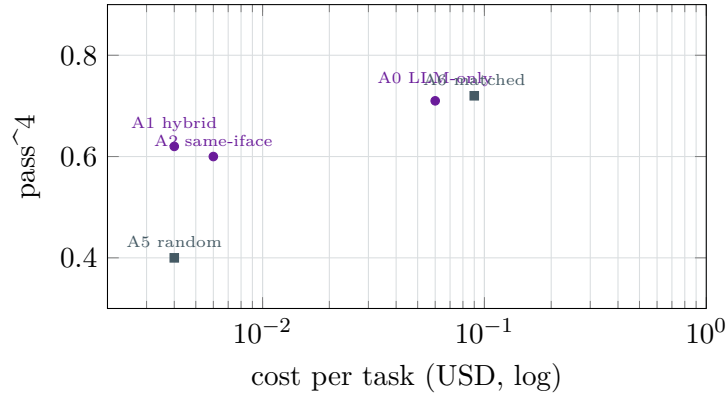


Figure 7.2.: Illustrative accuracy–cost plot. The question is whether the hybrid (A1) sits up-and-left of the baseline (A0) and beats the random control (A5). [Data: *Illustrative (synthetic data)*; arm definitions from docs/PLAN.md §9]

Oracle (ARM-A4)

perfect decisions, an upper bound: if even the oracle does not help, the chosen decision points are not the bottleneck.

Pre-registration

fix arms, metrics, thresholds, and seeds *before* looking at results; fit thresholds on a disjoint split; report bootstrap confidence intervals over ≥ 3 seeds.

Pitfall

Never use an LLM judge as sole ground truth for accuracy: it measures *agreement*, not correctness. One study reported 91.5% agreement with a frontier model, but with no human labels that is agreement, not accuracy [5].

Check yourself

An agent reports pass@8 of 0.95. Why might its pass⁴ be under 0.6, and which figure shows it? (Appendix C.)

Part III.

The system

8. Decision points DP1–DP7

The architecture is organized around seven recurring decision points. Each has a natural Jev primitive, some evidence it can be served, and a **risk class** that caps how much authority the engine may have.

ID	Decision	Primitive / evidence	Authority
DP1	intent / route	Choice; [IND] within a few pts of a frontier LLM	may act
DP2	tool pre-selection	Choice; [VENDOR] skill-suggestion cookbook	narrows only
DP3	action risk	Choice/Noul; [IND] 91.7% on 60 cases	advisory only
DP4	done / stuck	Noul/Choice; evidence thin	control flow only
DP5	observation triage	Noul per item; [VENDOR] RAG cookbooks	filter only
DP6	output verification	Choice/Noul; [IND] agreement not accuracy	routes to review
DP7	escalation	confidence from any DP	control flow

Figure 8.1.: The decision-point taxonomy. The MVP uses DP2 and DP4 (bold); DP3 is shadow-only, the rest are deferred. [Data: Reported by docs/ARCHITECTURE.md §3]

8.1. The design rule

Jev is a candidate only for decisions whose answer space is *closed*, whose relevant state fits in about 32k tokens, and whose consequences can be bounded by code. Authority scales inversely with risk.

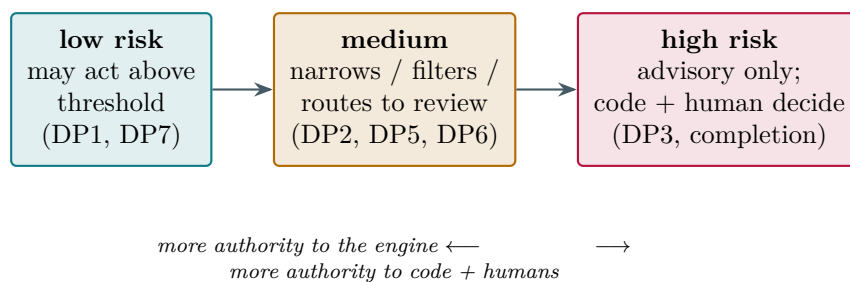


Figure 8.2.: Authority classes: the riskier the consequence, the less the engine may decide on its own. [Data: Illustrative (docs/ARCHITECTURE.md §3, §10)]

8.2. The two MVP decisions

DP2 tool pre-selection

a Choice over the permission-filtered catalog, always with a “`none_of_the_above`” escape hatch. High confidence exposes one tool; medium exposes the top-*k*; low exposes

8. Decision points DP1–DP7

the full catalog. The LLM still writes the arguments. It is advisory: the LLM may pick outside the top- k , and the disagreement is logged.

DP4 done / stuck

a Noul/Choice over a projection of the task, acceptance criteria, and the last few actions. “Done” with high confidence only *triggers* a deterministic completion verifier; it never ends the run by itself. v1 is scoped to “done” plus loop/repeat detection; “drift from goal” waits for an operational definition.

Pitfall

Options are permission-filtered per step, so the option *set* changes every call. “Fix the option order” really means “fix a stable sort key (by tool ID) that is invariant under filtering.” Otherwise calibration fitted on one option ordering will not transfer.

Check yourself

Why does DP2 narrow options rather than pick the final tool, and why does DP4 never declare success on its own? (Appendix [C](#).)

9. SDK architecture

The SDK extends the recommended architecture from the research (LLM-first, engine advisory): the LLM plans and writes; a swappable engine answers bounded questions; deterministic code owns safety, permissions, arithmetic, and verification.

9.1. Components

9.2. One loop iteration (MVP)

9.3. A single decision, in sequence

9.4. Confidence bands

9.5. Security boundary

Models advise; code authorizes. Untrusted inputs (user text, tool outputs) reach an advisory layer (the LLM proposal, DP3/DP5 screens), but the authoritative layer (permission filter, allow/deny lists, schema validation, human approval, sandbox) is deterministic. DP3 output can only *raise* the required approval level, never lower it.

9.6. Secrets: allow-list by reference

The SDK never redacts secrets out of a model-visible string (a deny-list always misses something). Instead, projections and logs carry secret *names*; the executor injects values at call time. A fuzz test smuggles a known token through every projection and asserts it never appears.

Listing 9.1: Engine swap is one line; the rest is unchanged.

```
1 dp2 = DecisionPoint(question=choice, engine=JevEngine(model="jev-1.13.0"),  
    policy=policy)  
2 # swap the engine, keep everything else:  
3 dp2 = DecisionPoint(question=choice, engine=LLMAdapterEngine(provider="..."),  
    policy=policy)
```

Check yourself

DP3 says a call is “read-only” with confidence 0.97, but the allow/deny list marks it destructive. What happens, and which principle decides? (Appendix C.)

9. SDK architecture

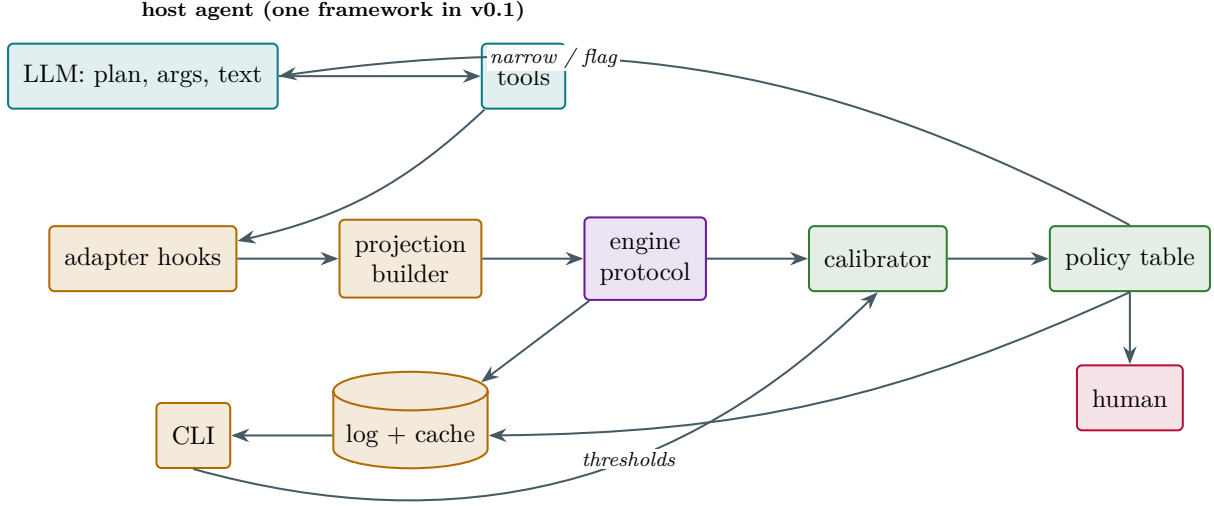


Figure 9.1.: SDK components. The engine is advisory; the policy narrows or escalates; code owns execution and safety. [Data: Illustrative (docs/ARCHITECTURE.md §4, docs/PLAN.md §4)]

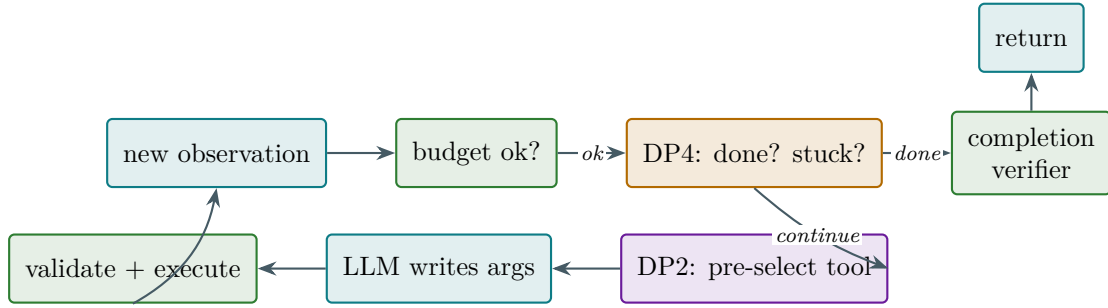


Figure 9.2.: The MVP loop: DP4 monitors progress, DP2 narrows the tool, the LLM writes arguments, and code validates and executes. [Data: Illustrative (docs/PLAN.md §5)]

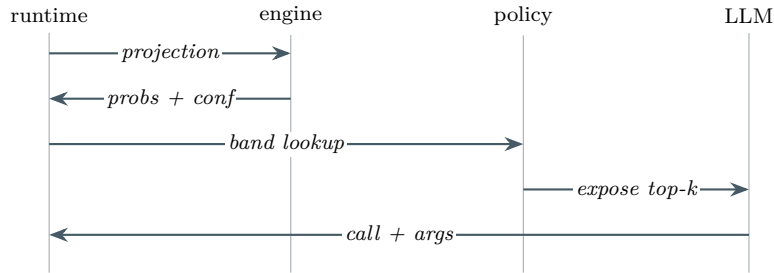


Figure 9.3.: Sequence for one DP2 decision: projection out, probabilities back, policy picks a band, the LLM fills arguments. [Data: Illustrative (docs/ARCHITECTURE.md §5.3)]

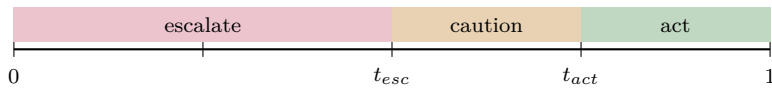


Figure 9.4.: The policy maps the gating statistic to three bands. Thresholds t_{esc} , t_{act} are fitted per DP, primitive, and model version. [Data: Illustrative (docs/ARCHITECTURE.md §6)]

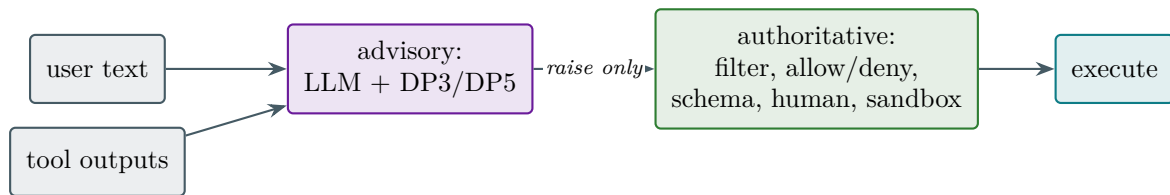


Figure 9.5.: Advisory vs authoritative layers. Injection is assumed; the engine is never the boundary. [Data: Illustrative (docs/ARCHITECTURE.md §10)]

10. Engineering practices

The SDK’s credibility depends on boring, rigorous engineering. These practices come straight from the plan review’s findings.

10.1. Record / replay cache

Every model call is cached, keyed by a content hash of the request. This gives deterministic offline re-runs (“one command regenerates every table, no API calls”) and makes results reproducible.

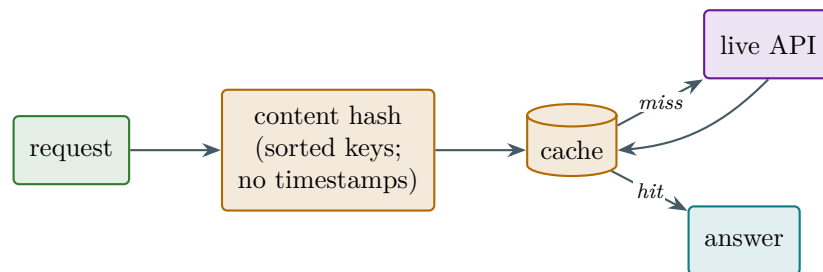


Figure 10.1.: Record/replay: on a miss, call the API and store; on a hit, replay. The hash excludes wall-clock and counters so it is stable across processes. [Data: Illustrative (docs/PLAN.md EF-11)]

Pitfall

Hash stability is easy to get wrong: Python’s `PYTHONHASHSEED`, dict/set ordering, and float formatting can all perturb serialization. The fix is canonical serialization (sorted keys, excluded volatile fields) plus a cross-process test with a randomized hash seed.

10.2. Version pinning

Every request pins an explicit Jev version and asserts the returned `model` field. The alias `jev-latest` was observed re-pointing mid-run, and thresholds are version-specific, so a version change must hard-fail and quarantine the run rather than silently gate on stale calibrators.

10.3. The test pyramid

Specific tests the plan requires:

- **Contract:** every engine returns the same typed shape; “confidence unavailable” is a first-class return (the LLM-adaptor has no logprobs).
- **Property:** token budget never exceeded; projection output deterministic; no secret ever appears in a projection or log.

10. Engineering practices

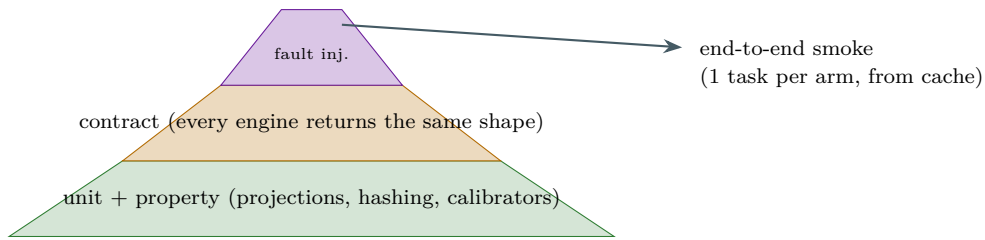


Figure 10.2.: The test pyramid for the harness. Contract tests keep engines interchangeable; fault injection proves the fallback path. [Data: *Illustrative (docs/PLAN.md §7.3)*]

- **Fault injection:** fallback fires on 429/529/timeout, malformed JSON, and out-of-range Choice indices; the fallback reconstructs *full* context so it is not handicapped by the lossy projection.
- **Fail-closed:** a state-machine test proves no side-effecting call reaches execution without the deterministic gate, and DP outputs may only feed narrow/deny/escalate, never allow/elevate.
- **Drift guard:** an injected distribution shift must trigger a fail-safe that widens bands or escalates.

Why it matters here

Two subtle correctness traps the reviewer caught: (1) if the fallback feeds the LLM the *lossy* projection while the baseline sees full context, a fallback-heavy run looks worse for reasons unrelated to the engine, biasing the whole evaluation; (2) the “cost per successful task” denominator (“decision-equivalent tokens”) must be defined and pre-registered, or the headline economic claim is unfalsifiable.

Check yourself

Why must the fallback path rebuild full context instead of reusing the projection? What would break in the evaluation if it did not? (Appendix C.)

Part IV.

Evidence so far

11. Evidence so far

This chapter summarizes what independent testers found in the days after Jev’s launch. All of it is early, small, and unreproduced, so treat it as indicative, not settled. We have run no experiments of our own yet, so there are no [MEASURED] labels here.

11.1. Accuracy: competitive on short tasks, weak on long ones

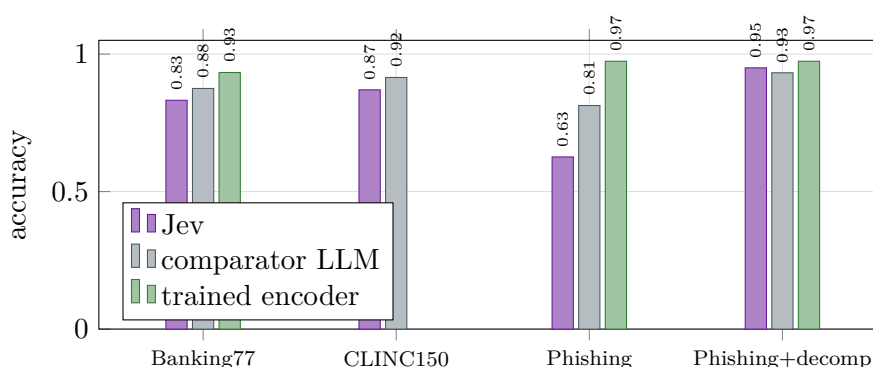


Figure 11.1.: Jev is within a few points of the comparator LLM on short-input intent classification, clearly below on whole-document phishing, and recovers most of the gap when the judgment is decomposed. The comparator is the frontier model GPT-5.6 Terra on Banking77/CLINC150 and Claude Haiku 4.5 on phishing; the encoder is bge-small+logistic regression on Banking77 and a 4B LoRA model on phishing (no published CLINC150 encoder number, so that bar is omitted). A trained in-domain encoder tends to win when labels exist. [Data: Reported by *jev-baselines-eval* and *jev-phishing-bench*] [1, 7]

11.2. Latency: fast, but with tails under load

Independent medians cluster around 0.24–0.9s per call depending on location and concurrency, with rare tails of tens of seconds at high concurrency [10]. Fast enough to ask every step; the tails are why the SDK must be complete without Jev.

11.3. Calibration

The out-of-distribution ECE result (Figure 5.2) is the single most important calibration finding: near the noise floor in distribution, about $4.4\times$ the floor out of distribution, and type-dependent (Choice/Score overconfident, Noul underconfident) [15]. Recalibration with a few hundred labels brings ECE back toward 0.01 [2].

11. Evidence so far

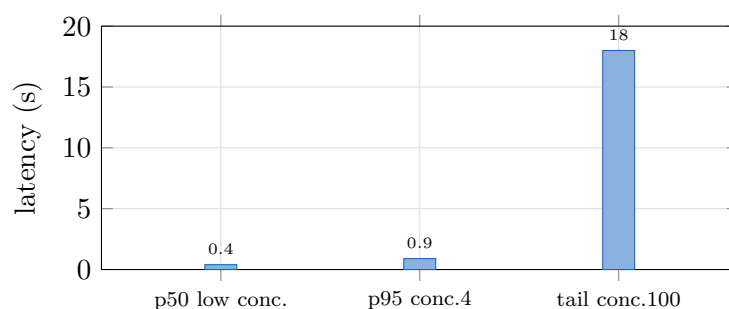


Figure 11.2.: Illustrative latency profile: sub-second typical, multi-second tails under heavy concurrency. [Data: *Illustrative*; ranges reported by Aman Kumar] [10]

ID	Hypothesis (abridged)	Status
H0	separating decision from reasoning helps	partially supported, unproven end-to-end
H1	Jev picks the right tool (DP2) cheaply	plausible, vendor-supported, unmeasured on real catalogs
H2	inserting it lowers cost/task w/o hurting pass ^k	open — the MVP hypothesis
H3	Jev judges progress (DP4)	untested
H4	probabilities gate well after recalibration	partially supported
H5	the benefit is specific to Jev, not typed points	open; ARM-A2 exists to test it

Figure 11.3.: Hypothesis status at the start of the build. The gates G0/G1/G2 are designed to move these. [Data: *Reported by docs/DECISION_LOG.md §1*]

11.4. Where the hypotheses stand

Why it matters here

The honest expected outcome, from the whole evidence base, is “cheaper decisions at equal or slightly lower accuracy, useful after recalibration.” The project is built so that this result, if it holds, is a *success*: it is exactly the controlled evidence nobody has published, and the SDK ships it reproducibly.

Check yourself

Which single hypothesis, if it failed at gate G2, would most change the product story, and why? (Appendix C.)

Part V.

Build log

12. Build log

New dated entries are appended here as tasks finish, so this book doubles as a project diary.

Entry 0 — Research and planning (2026-09-20 to 2026-09-27)

2026-09-20. Research phase completed: four documents produced (RESEARCH, ARCHITECTURE, IMPLEMENTATION_PLAN, DECISION_LOG). Research gate: *MODIFY*. Key conclusions: Jev is fast and cheap on bounded closed-set judgments over compact state; its accuracy sits at or slightly below cheap LLMs and below trained encoders on hard tasks; its calibration is imperfect out of distribution but recalibratable; the missing thing is controlled evidence, not another integration.

2026-09-27. Product phase began. Decisions recorded:

- **D11** — the product is an open-source Python SDK (supersedes the harness-only scope D1 and the “defer a library” decision D9; closes D8). The install name is set in D17.
- **D12** — docs/PLAN.md is the operational source of truth.
- **D13** — Python 3.12 via uv.
- **D14** — this LaTeX book, built with Tectonic, with a tracked master PDF.
- **D16** — the evidence artifact is elevated to a co-headline deliverable.

Work done this session:

- Git repository initialized; the four research docs committed as a baseline.
- Office-hours design doc written and passed an independent 3-round spec review ($7 \rightarrow 8 \rightarrow 8/10$, converged).
- Plan review (CEO, Eng, DX) run with independent reviewers; Codex was unavailable (expired token), so this was subagent-only. Findings folded into PLAN.md as engineering findings EF-01–EF-17 and DX findings, plus a strategic “User Challenge” (Q17) surfaced for the user: consider making the evidence/benchmark the headline.
- docs/PLAN.md written (16 sections) and DECISION_LOG.md amended.
- This learning book created and built with Tectonic; master PDF generated.

Not done this session (by design): SDK code, API keys, experiments, publishing. Those begin at milestone M0.6 onward and need the human touchpoints listed in PLAN.md §10.

Entry 1 — Package name (2026-09-27)

2026-09-27. Adopted the package name `arbiter` (ARBITER: Agent Runtime for Bounded Inference, Triage, Evaluation & Routing). Decision D17 supersedes the working name in D15 and resolves Q16 for the install name. The import path is `arbiter`. License and publishing stay at touchpoint H8.

Key idea

The most important single artifact from the research phase is the “jaggedness” table (Figure 4.2): nine documented failure modes that the entire architecture is designed around.

Part VI.

Appendices

A. Glossary

Agent loop

observe, decide, act, observe again; the cycle an LLM agent runs.

ARBITER

Agent Runtime for Bounded Inference, Triage, Evaluation & Routing. The product. The Python package and import name is `arbiter` (D17).

AUROC

area under the ROC curve; here, how well a confidence score ranks correct answers above incorrect ones (0.5 = chance, 1.0 = perfect).

Brier score

mean squared error between predicted probabilities and outcomes; a proper scoring rule for probabilistic predictions.

Calibration

the property that predictions stated at probability p are correct a fraction p of the time, measured over groups.

Cascade

try a cheap model first, escalate to an expensive one on low confidence.

Choice

a Jev primitive: pick one of up to 255 options; returns per-option probabilities and a confidence.

Confidence (Jev field)

a statistic of the probability distribution; formula undisclosed; independently found weaker than max probability for gating.

Conformal prediction

builds a set guaranteed to contain the truth with chosen probability, using a calibration set.

Coverage

the fraction of decisions a policy acts on automatically at a given threshold.

Decision point (DP)

a bounded, closed-set decision in the agent loop (DP1–DP7).

ECE

expected calibration error; average gap between confidence and accuracy across probability bins.

Engine

the swappable component that answers a decision question (Jev, LLM-adaptor, classifier, rules, oracle, random).

Gate (G0/G1/G2)

a go/modify/stop checkpoint with pre-registered criteria.

A. Glossary

Isotonic regression

a monotonic post-hoc recalibrator that maps stated probability to observed frequency.

Noul

a Jev primitive: the probability that a statement is true; no separate confidence.

Noise floor

the small nonzero ECE a perfectly calibrated model shows on finite data.

pass@k / pass[^]k

success in at least one of k tries / in all k tries; the latter measures reliability.

Pareto frontier

the set of systems not beaten on both cost and accuracy at once.

Policy table

the deterministic rules mapping a gating statistic to act/narrow/escalate.

Projection

a token-budgeted, secret-free view of agent state sent to the engine.

Record/replay cache

content-hashed store of model responses enabling deterministic offline re-runs.

Router

sends each query to the cheapest capable model, deciding before paying.

Score

a Jev primitive: a position on an ordered 2–10 level scale.

System One model

TypeSafe’s term for a fast, bounded-judgment model (after Kahneman’s System 1); Jev is one.

Temperature scaling

a one-parameter post-hoc recalibrator on logits.

Trajectory-level calibration

calibration measured across checkpoints of a whole run, not just per step.

B. Formula sheet

Calibration

Expected Calibration Error (binned into B bins, n_b items in bin b , N total):

$$\text{ECE} = \sum_{b=1}^B \frac{n_b}{N} |\text{acc}(b) - \text{conf}(b)|.$$

Brier score (binary outcome $y_i \in \{0, 1\}$, predicted probability p_i):

$$\text{BS} = \frac{1}{N} \sum_{i=1}^N (p_i - y_i)^2.$$

Temperature scaling (logits z , learned scalar $T > 0$):

$$\hat{p} = \text{softmax}(z/T).$$

$T > 1$ softens an overconfident model; $T < 1$ sharpens an underconfident one.

Reliability

pass@ k and **pass $\hat{k}$** for per-attempt success probability p :

$$\text{pass}@k = 1 - (1 - p)^k, \quad \text{pass}\hat{k} = p^k.$$

Economics

Cost per successful task (all model + fallback calls, in USD):

$$\text{CPST} = \frac{\text{total USD across all calls}}{\text{number of successful tasks}}.$$

Jev input cost at list price (\$0.042 per million input tokens, output free):

$$\text{cost} = 0.042 \times \frac{\text{input tokens}}{10^6} \text{ USD}.$$

Worked examples

Worked example

For $p = 0.7$ and $k = 4$: $\text{pass}@4 = 1 - 0.3^4 = 0.9919$, while $\text{pass}\hat{4} = 0.7^4 = 0.2401$. The gap is why we report $\text{pass}\hat{k}$ (Figure 7.1). [Data: Exact (computed from formula)]

B. Formula sheet

Worked example

20,000 decisions at 1,500 input tokens each: $0.042 \times (20000 \times 1500)/10^6 = 0.042 \times 30 =$
\$1.26. [*Data: Exact (computed from list price)*]

C. Answers to “Check yourself”

1. (Ch. 1) An answer can be right by luck or on easy items while the stated confidence does not track correctness across the distribution. You catch it by measuring calibration (a reliability diagram and ECE against the noise floor), not just accuracy.
2. (Ch. 2) The record/replay cache makes every result reproducible offline with no API calls, so at gate review you can regenerate every table and audit exactly what was measured. It also removes network flakiness and cost from the build-test loop.
3. (Ch. 3) Guaranteed: valid output shape and a valid schema for tool arguments. Not guaranteed: a probability distribution over which tool was chosen. The decision layer adds that calibrated probability.
4. (Ch. 4) (1) Adversarial content is a documented Jev weakness, so it can be fooled by a benign-looking wrapper. (2) A saturated confidence of 0.99 can be wrong, and safety-critical authorization must be deterministic; code and allow/deny lists, not a probability, decide.
5. (Ch. 5) 92% accuracy with ECE 0.15 means the probabilities are badly miscalibrated, so “confidence ≥ 0.9 ” may include many wrong items. Inspect the reliability diagram (Figure 5.1) and the ECE-vs-floor bars (Figure 5.2) first, then recalibrate before gating.
6. (Ch. 6) Because Jev is so cheap it can be queried *before* paying for the LLM, like a pre-generation router; a cascade instead pays the cheap model first and only then decides, which wastes the cheap call when it defers.
7. (Ch. 7) pass@8 counts a single lucky success out of eight, while pass[^]8 requires success in all eight; an unreliable per-attempt rate (say 0.7) gives $0.7^8 \approx 0.058$. See Figure 7.1.
8. (Ch. 8) DP2 narrows because Jev cannot write tool arguments (no generation) and its choice is advisory; keeping the LLM in the loop preserves correctness. DP4 never declares success alone because “done” is safety-relevant and must be confirmed by a deterministic completion verifier.
9. (Ch. 9) The call is treated as destructive (most conservative wins); DP3 output can only raise the required approval level, never lower it. The principle is “the engine advises, code authorizes.”
10. (Ch. 10) If the fallback reused the lossy projection while the baseline saw full context, fallback-heavy runs would look worse for reasons unrelated to the engine, biasing the whole cost/quality comparison. Rebuilding full context keeps the arms comparable.
11. (Ch. 11) H5: if ARM-A2 (an LLM answering the same typed questions) matches ARM-A1, the benefit is from typed decision points, not Jev specifically, so the product story shifts from “Jev” to “calibrated typed decisions, with Jev as the cheap option.”

D. Source map

Where each chapter's material comes from in the frozen research docs and the operational plan. Use this to trace any claim back to its origin.

Book chapter	Primary sources
Ch. 1 What we are building	docs/PLAN.md §2; design doc; RESEARCH §5.5, §10
Ch. 2 The roadmap	docs/PLAN.md §6–7; IMPLEMENTATION_PLAN §2–3
Ch. 3 LLM agents	RESEARCH §5.1; ARCHITECTURE §1
Ch. 4 Jev / System One	RESEARCH §1, §4; TypeSafe docs (models, primitives, jaggedness)
Ch. 5 Calibration	RESEARCH §4.3; jev-ood-calibration; Jev-Calibration; Guo et al.
Ch. 6 Routers/cascades	RESEARCH §5.2; RouteLLM; FrugalGPT; cascade-deferral
Ch. 7 Evaluating agents	RESEARCH §9; tau-bench; AI Agents That Matter
Ch. 8 Decision points	ARCHITECTURE §3
Ch. 9 SDK architecture	ARCHITECTURE §4–10; docs/PLAN.md §4–5, §7.4
Ch. 10 Engineering practices	docs/PLAN.md §7.3 (EF-01..EF-17); ARCHITECTURE §9
Ch. 11 Evidence so far	RESEARCH §4.2–4.3; independent studies (see bibliography)
Ch. 12 Build log	docs/DECISION_LOG.md §11; this session

Why it matters here

Data-quality note: RESEARCH.md lines 159–169 contain character-encoding corruption, and three dollar figures there are unrecoverable. This book does not cite those three values; they will be re-verified against the source cookbooks at task PT-2.1 before any use.

Bibliography

- [1] anisselbd. jev-phishing-bench: phishing detection benchmark. <https://github.com/anisselbd/jev-phishing-bench>, 2026. Jev 62.6% vs Claude Haiku 4.5 81.3% on 2,000 emails; decomposition + logistic regression reaches 95.0%. Accessed 2026-09-20.
- [2] AnthusAI. Jev-calibration: recalibration with isotonic regression. <https://github.com/AnthusAI/Jev-Calibration>, 2026. 8,801 sentiment examples; isotonic regression reduces ECE to 0.008–0.016; 100 labels reach ECE 0.048. Accessed 2026-09-20.
- [3] Lingjiao Chen, Matei Zaharia, and James Zou. Frugalgpt: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023. <https://arxiv.org/abs/2305.05176>.
- [4] Convai Innovations. Laya: an open-weights system one model. <https://huggingface.co/convaiinnovations/laya>, 2026. 421M ModernBERT-large; same three primitives; Apache-2.0. Accessed 2026-09-20.
- [5] Good Start Labs. Verification is the bottleneck. <https://goodstartlabs.com/research/verification-is-the-bottleneck>, 2026. 6,003 rubric checks; 91.5% agreement with Claude Fable 5.1; agreement, not human ground truth. Accessed 2026-09-20.
- [6] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning (ICML)*, 2017. Temperature scaling; expected calibration error. <https://arxiv.org/abs/1706.04599>.
- [7] ickma2311. jev-baselines-eval: pre-registered intent-classification baselines. <https://github.com/ickma2311/jev-baselines-eval>, 2026. Banking77 0.832, CLINC150 0.870; encoder 0.933 on Banking77; confidence saturates at 1.0. Accessed 2026-09-20.
- [8] Wittawat Jitkrittum et al. When does confidence-based cascade deferral suffice? *arXiv preprint arXiv:2307.02764*, 2023. <https://arxiv.org/abs/2307.02764>.
- [9] Sayash Kapoor et al. Ai agents that matter. *arXiv preprint arXiv:2407.01502*, 2024. Accuracy–cost Pareto frontiers; retry baselines. <https://arxiv.org/abs/2407.01502>.
- [10] Aman Kumar. Jev, measured: classifier or filter? <https://amankumar.ai/blogs/jev-measured>, 2026. Latency and accuracy measurements; $P < 0.1$ was 99.7% correct on 82% of items. Accessed 2026-09-20.
- [11] LangChain. Typesafe integration — langchain documentation. <https://docs.langchain.com/oss/python/integrations/providers/typesafe>, 2026. TypeSafeClassifier, ModelRouterMiddleware, AutoModeMiddleware. Accessed 2026-09-20.
- [12] Isaac Ong et al. Routellm: Learning to route llms with preference data. *arXiv preprint arXiv:2406.18665*, 2024. <https://arxiv.org/abs/2406.18665>.
- [13] Pydantic. Typesafe model — pydantic ai documentation. <https://pydantic.dev/docs/ai/models/typesafe/>, 2026. Jev as a model with FallbackModel. Accessed 2026-09-20.

- [14] Allen Z. Ren et al. Robots that ask for help: Uncertainty alignment for large language model planners (knowno). *arXiv preprint arXiv:2307.01928*, 2023. Split conformal prediction over option logprobs. <https://arxiv.org/abs/2307.01928>.
- [15] scienthoon. jev-ood-calibration: out-of-distribution calibration of jev. <https://github.com/scienthoon/jev-ood-calibration>, 2026. In-distribution ECE near noise floor; OOD ECE 0.107 vs 0.024 floor; type-dependent miscalibration. Accessed 2026-09-20.
- [16] TypeSafe AI. Jev 1.13 jaggedness (known issues) — typesafe ai documentation. <https://docs.typesafe.ai/model-jaggedness/jev-1.13>, 2026. Nine documented failure modes with workarounds. Accessed 2026-09-20.
- [17] TypeSafe AI. Models — typesafe ai documentation (jev). <https://docs.typesafe.ai/models>, 2026. jev-1.13.0; \$0.042 per million input tokens, output free; 64k tokens/request, 32k for state + longest question. Accessed 2026-09-20.
- [18] TypeSafe AI. Primitives: Choice, score, noul — typesafe ai documentation. <https://docs.typesafe.ai/primitives>, 2026. Choice ≤ 255 options; Score 2–10 levels; Noul probability. Accessed 2026-09-20.
- [19] TypeSafe AI. System one — typesafe ai documentation. <https://docs.typesafe.ai/concepts/system-one>, 2026. Text-only input; does not generate text. Accessed 2026-09-20.
- [20] Shunyu Yao et al. tau-bench: A benchmark for tool-agent-user interaction. *arXiv preprint arXiv:2406.12045*, 2024. Introduces pass[^]k. <https://arxiv.org/abs/2406.12045>.